Architectural Decisions

MathVerity uses a modular monolith. Server-side code determines mathematical correctness, while external AI provides only restricted hints. The decisions below define module boundaries, assessment and data access.

Figure 1 shows the draft component structure; the sections below explain the decisions behind it.

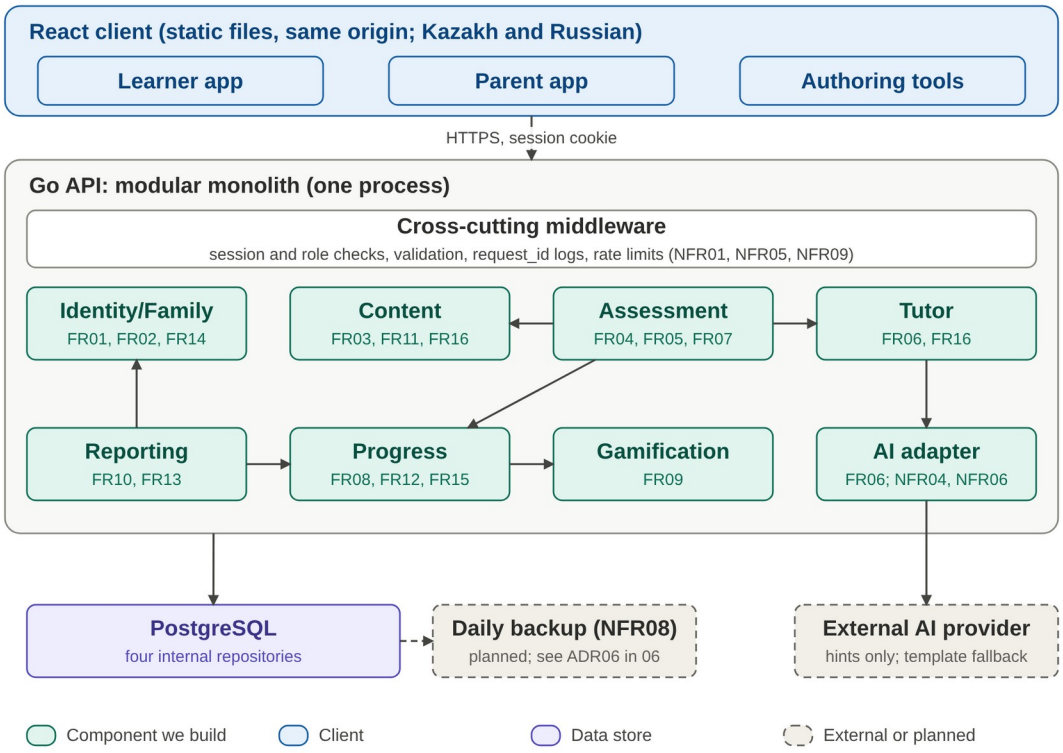


Figure 1. Component diagram of the MathVerity API (draft). Arrows show the main calls between modules.

Context and Constraints

The system serves learners and linked parents. Content is authored through a restricted internal capability of the two developers, who have no access to identifiable family results. The AI provider is an external dependency; if it fails, a static template supplies the hint. PostgreSQL is internal storage rather than an external system on the context diagram. The supervisor participates in development and does not automatically receive an administrative role over children’s data.

The React client is delivered as static files and calls the Go API over HTTPS on the same origin. A single Go process contains Identity/Family, Content, Assessment, Tutor, Progress, Gamification and Reporting modules. Assessment is the source of truth for answer correctness; Progress derives diagnostic status from stored evidence. Tutor cannot alter grades, family links or rewards.

ADR01 Modular Monolith

Decision: use a modular monolith. Rationale: a shared transactional learning cycle and a small team. Consequence: deployment is simpler, but module boundaries must be enforced. Revisit when hints or analytics require independent scaling.

ADR02 Deterministic Assessment

Code determines mathematical correctness. Fractions use exact rational arithmetic with input limits. Decimals use an exact representation, or a defined tolerance only where the item permits it. The LLM cannot change the final score. Revisit for new item types after reference examples have been assembled.

ADR03 Independent Assessment

Independent assessment is separate from learning. It uses new variants, a server-side prohibition on AI and hints, a separate event and recorded assessment conditions. This reduces confusion between assistance and assessment but cannot exclude external help.

ADR04 Explainable Skill Status

Do not use an opaque “percentage of understanding”. Reports show statuses and supporting evidence. ML predictions may be introduced only after calibration and comparison against a baseline model.

ADR05 Sessions and Access Control

Use one origin for client and API, a server-side session in an HttpOnly Secure SameSite cookie, CSRF protection for modifying requests and role and object checks on every resource. This reduces browser authentication complexity. Actual settings must be verified through security tests as well as architectural design.

ADR06 Backup and Recovery

Decision: take a daily PostgreSQL backup in the Docker environment and restore it to a test copy. Rationale: the recovery requirement allows at most 24 hours of data loss and a restore within 60 minutes. Consequence: a separate restore test is required, and deleted family profiles are handled by the documented deletion procedure. Revisit if the pilot needs a shorter data-loss window.

AI Adapter Boundaries

Browser-supplied results, XP, roles and activity are untrusted. The server holds private answers and rubrics. User text is untrusted data, not model instructions. In the MVP, the LLM receives only item context, a numeric answer, the requested language and structured skill, error and step codes; children’s free text is not sent. AI responses are constrained by the hint_text schema, pedagogical rules, length limits and filtering; violations trigger a template fallback. A simple string check cannot prove that every response avoids giving away the solution: a reference set and a manual check by the team are needed.

AI receives no tools, database access, URLs to read, tokens or child identifiers. Assessment calls Content and Tutor; Reporting calls Family and Progress. Only internal repositories for Identity/Family, Content, Assessment and Progress/XP access PostgreSQL [T01, T02]; Tutor has no direct database access. Visible-activity windows (FR13) are stored through the Progress/XP repository. The provider key stays on the server. Rate limits apply per account and to the global budget. Provider selection remains open until Kazakh- and Russian-

language hint quality, cost and age-related and contractual conditions have been reviewed.

Risks and Mitigations

The main architectural and delivery risks are listed below with their planned mitigations.

Table 1. Architectural and delivery risks

No.	Risk	Likelihood	Impact	Mitigation
1	**AI provider is unavailable or slow**	M	L	A static hint template is shown when the call fails or times out; the template mode works without any provider.
2	**A hint gives away the full solution**	M	H	Output is limited by schema, length and filtering; a reference set is checked manually by the team; independent assessment forbids AI and hints.
3	**Hint cost exceeds the budget**	M	M	Rate limits per account and a global budget; template mode is the default baseline.
4	**Provider terms do not suit data about children**	L	H	No child identifiers or free text are sent; the provider is chosen only after age-related and contractual conditions are reviewed.
5	**XP is awarded twice or state becomes inconsistent**	L	M	Attempt, diagnostic result and XP are written in one transaction; an idempotency key prevents duplicate awards.
6	**Unauthorised access to a child’s data**	L	H	Server-side session in an HttpOnly Secure SameSite cookie, CSRF protection, and role and object checks on every resource; verified by security tests.
7	**Module boundaries erode inside the monolith**	M	M	Only four internal repositories access PostgreSQL; the two developers review each other’s changes against the module rules.
8	**The scope is too large for a two-person team**	M	H	The MVP is limited to four topic blocks and eight skills; features outside this scope are deferred.

L = low, M = medium, H = high. Likelihood and impact are the team’s initial qualitative estimates and will be reviewed as the implementation progresses.

Versions and Local Environment

Dependency versions and Docker image digests [T05] must be pinned during implementation. The latest tag is not used as a reproducible project version. A local PostgreSQL configuration is not a production deployment.

The local example specifies PostgreSQL 17 with patch version 17.11 [T15]. Availability of the particular Docker tag has not been tested by running it; the digest should be recorded after the first successful image pull.

References

[T01] Go. Accessing relational databases and transactions. https://go.dev/doc/database/ ; https://go.dev/doc/database/execute-transactions

[T02] PostgreSQL. Constraints and JSON types. https://www.postgresql.org/docs/current/ddl-constraints.html ; https://www.postgresql.org/docs/current/datatype-json.html

[T05] Docker Compose. https://docs.docker.com/compose/

[T15] PostgreSQL 17.11 release notes, 13 August 2026. https://www.postgresql.org/docs/release/17.11/